Esercizio Laboratorio 2:

Progetto: "Smart ICU - Sistema di Monitoraggio Terapia Intensiva"

Lo Scenario: Stai sviluppando il software per il reparto di Terapia Intensiva di un ospedale. Ogni letto ha un "Monitor Paziente" (il Client) che trasmette i parametri vitali alla "Stazione Infermieri" centrale (il Server).

1. **Il Canale TCP (Porta 9000 - "Stato Paziente"):** Usato per registrare il ricovero del paziente, confermare l'assegnazione del letto e inviare gli allarmi medici critici (`CODE_RED`). È vitale che questo canale non perda pacchetti.
2. **Il Canale UDP (Porta 9001 - "Parametri Vitali"):** Il monitor spara ogni secondo i valori in tempo reale: Battito Cardiaco (BPM) e Saturazione Ossigeno (SpO2). Se un pacchetto UDP si perde, pazienza, un secondo dopo arriverà quello nuovo.

---

1. La Stazione Infermieri (SERVER)

Il Server gestisce una lista concatenata globale dei pazienti attualmente ricoverati: `ID_Paziente`, `Socket_TCP`, protetta rigorosamente da un `pthread_mutex_t` (ora sai quanto è importante!).

Il Server ha queste anime:

- **Thread TCP_Listener (Accettazione):** Sta in `accept()`. Quando un Monitor si connette, riceve un pacchetto `PATIENT_ADMIT` con l'ID del paziente. Lo aggiunge alla lista globale (bloccando il Mutex!), gli risponde con `ADMIT_ACK`, e si mette in una `recv()` bloccante per monitorare se il letto viene staccato.
- **Thread UDP (Il Cuore del Sistema):** Un singolo thread bloccato su una `recvfrom()`. Riceve a raffica i pacchetti `VITALS_UPDATE` da tutti i letti.

- Stampa a video: `[LETTO 3] BPM: 85 | SpO2: 98%`.
- **LA SFIDA (Emergenza Medica):** Se il thread UDP legge che il Battito (BPM) scende sotto 40 o l'Ossigeno scende sotto il 90%, scatta l'allarme. Il thread UDP blocca il Mutex, cerca il `Socket_TCP` di quel paziente, e gli spara dentro un pacchetto TCP `CODE_RED`. Poi, stampa a tutto schermo: `[!!! ALLARME ROSSO !!!] ARRESTO CARDIACO LETTO X!`.

2. Il Monitor Paziente (CLIENT)

All'avvio, passi da terminale l'ID del paziente (es. `./monitor 5`).

1. **Fase di Ricovero (TCP):** Si connette alla porta 9000. Invia `PATIENT_ADMIT` e aspetta la risposta `ADMIT_ACK` dal server.
2. **Sdoppiamento (Multi-Thread):**
   - **Thread Sensori (UDP):** Gira in un `while(1)`. Ogni secondo (`sleep(1)`), genera valori:
     - `BPM`: un numero casuale tra 60 e 100. *Ma attenzione*, fai in modo che ci sia una piccola probabilità (es. 5%) che il battito crolli improvvisamente a 30!
     - `SpO2`: un numero casuale tra 95 e 100. Invia questi dati al Server via UDP (Porta 9001).
   - **Thread Allarme (TCP):** Gira in un `while(1)` con una `recv()` bloccante sul socket TCP.
     - Se riceve `CODE_RED`, stampa a caratteri cubitali `[EMERGENZA] SOMMINISTRARE ADRENALINA! Rianimazione in corso...`. Chiude il socket, uccide il processo (`exit(0)`) simulando che il monitor viene staccato per l'intervento dei medici.

La "Trappola da Esame" (L'Insidia)

La vera difficoltà di questo esercizio rispetto a quello della Battle Royale è la **pulizia della memoria (Garbage Collection)**. Cosa succede se un monitor viene spento normalmente dall'infermiere (chiusura del Client tramite `CTRL+C` o

disconnessione)? La `recv()` bloccante del thread TCP sul Server restituirà `<= 0`. A quel punto, tu *devi* rimuovere quel paziente dalla lista concatenata usando il Mutex. Se non lo fai, e un secondo dopo un pacchetto UDP in ritardo fa scattare un falso allarme, il thread UDP cercherà di mandare un `CODE_RED` a un socket TCP che è già stato chiuso, facendo crashare l'intero Server dell'ospedale!

Struttura Pacchetto (`HospitalProtocol.h`)

```
#define PORT_TCP_HOSPITAL 9000
#define PORT_UDP_HOSPITAL 9001

#define PATIENT_ADMIT 1
#define ADMIT_ACK 2
#define VITALS_UPDATE 3
#define CODE_RED 4

typedef struct {
    int type;           // Tipo di pacchetto
    int patient_id;     // ID del paziente/letto
    int bpm;            // Battiti per minuto (UDP)
    int oxygen;         // Saturazione ossigeno % (UDP)
} MedPacket;
```